PERSONALIZED CYBERSECURITY LEARNING ROADMAP

Beginner foundations → independent practical skills → intermediate readiness

Prepared around your current situation

Third-semester cybersecurity student • 1–2 hours/day • 6–12 month horizon

macOS on Apple Silicon • 16 GB RAM • 20–50 GB available storage • UTM with Kali Linux CLI

Your guiding principle
Learn the mechanism, observe the evidence, practise safely, explain the result, then repeat without a walkthrough.

This guide prioritizes free resources, prerequisite-based learning, retention, and useful portfolio work—not badges or artificial contribution streaks.

Resource access and pricing notes checked against official pages on 14 October 2026. Free tiers and platform policies can change; verify before planning around any specific room or module.

Table of Contents

1. Your Starting Point and the Destination
2. The Roadmap at a Glance
3. Sustainable Weekly Schedule
4. Repeatable Study Method
5. Phase 0 — Setup and Baseline
6. Phase 1 — Linux and Operating-System Fundamentals
7. Phase 2 — Networking and Traffic Reasoning
8. Phase 3 — Bash, Python, Git, and Documentation
9. Phase 4 — Security Principles, Reconnaissance, and Enumeration
10. Phase 5 — Web Technologies and Application Security
11. Phase 6 — Windows, Privilege Concepts, and Defensive Fundamentals
12. Phase 7 — Integration, CTF Independence, and Capstone
13. Resource Directory and Free-Access Notes
14. GitHub Portfolio Structure
15. Troubleshooting and Getting Unstuck
16. Weekly Review, Tracker, and Maintenance
17. Final Checklist and First Seven Days
18. Sources, Verification, and Important Caveats

1. Your Starting Point and the Destination

You have some real hands-on exposure: basic Linux and networking, a few TryHackMe enumeration/basic-pentesting labs, several pwn.college dojos, prior SQL injection labs, and beginner Python/Git. Treat this as a restart for structure—not a reset to zero.

Starting assumptions

- Linux: basic command-line familiarity; currently studying HTB Academy Linux Fundamentals.
- Networking: familiar with basics, but protocols and packet flow need a systematic refresh.
- Windows: beginner; build operating-system and administration literacy before Windows privilege escalation.
- Bash: almost no scripting; learn small automation after command-line fluency.
- Python: basic; use it to understand data handling and automate small, explainable tasks.
- Web security: SQL injection is a strength worth rebuilding with a repeatable methodology.
- Hardware: Apple Silicon, 16 GB RAM, 20–50 GB free storage. Prefer browser labs and one compact local VM.

What 'intermediate foundation' means here

By the end, you should be able to describe a system and its network traffic, inspect common services, use a repeatable assessment workflow on authorized targets, explain several common web vulnerabilities, write small scripts, read basic Linux/Windows evidence, document findings clearly, and solve beginner CTF tasks without step-by-step instructions.

This roadmap does not promise job readiness or mastery of every specialization. It gives you a durable base and a reasoned next step.

2. The Roadmap at a Glance

Phase / target window	Focus	Evidence you should produce
0 • Setup & baseline Week 1	Audit environment, set up notes/repo, baseline skills	A working lab, learning log, baseline self-test
1 • Linux & systems Weeks 2–6	Files, permissions, processes, services, shell, OS basics	Linux command notebook + troubleshooting exercises
2 • Networking Weeks 7–11	TCP/IP, DNS, ports, routing, HTTP/TLS, packet reasoning	Explain a connection; interpret DNS/HTTP and scan output
3 • Scripting & Git Weeks 12–15	Bash/Python basics, Git workflow, safe automation	Two small scripts and a clean learning repository
4 • Security workflow Weeks 16–20	Recon, enumeration, service reasoning, evidence, reporting	A scoped assessment report from an authorized lab
5 • Web security Weeks 21–27	HTTP, sessions, auth, SQLi, XSS, access control, APIs	Re-solved PortSwigger labs + remediation notes
6 • Host & defense Weeks 28–34	Windows basics, Linux/Windows privilege concepts, logs, traffic	Mini investigation and host-hardening checklist
7 • Integration Weeks 35–42	Beginner CTFs, capstone, review and specialization sampler	Portfolio-ready capstone and readiness assessment

This is a flexible sequence, not a deadline contract. At 1–2 hours/day across seven days, plan roughly 7–10 hours/week of focused work after allowing for rest and university pressure. If a phase needs more time, extend it rather than skipping prerequisites.

Essential vs supplementary

Priority	Topics
Essential	Linux CLI and permissions; networking; HTTP; basic Windows concepts; Bash/Python fundamentals; Git; security methodology; common services; SQLi/XSS/access control; logs and safe lab practice.

Priority	Topics
Useful supplementary	Wireshark filters; regular expressions; basic SQL; JSON/API literacy; simple Docker usage; Markdown automation; threat modelling.
Postpone until foundations are steady	Advanced exploit development, kernel exploitation, complex Active Directory attack chains, malware reverse engineering, cloud exploitation, advanced evasion, and large multi-VM ranges.

3. Sustainable Weekly Schedule

Your classes are Sunday, Monday, Tuesday, and Thursday, with you home after about 3 PM. The schedule below keeps class days lighter and places longer lab work on more flexible days. Shift the clock times to match commute, meals, and assignments.

Day	Suggested session	Purpose
Sunday (class)	30–45 min	Review notes + one active-recall quiz; no pressure to finish a lab
Monday (class)	45–60 min	Continue current theory module and capture 3 key ideas
Tuesday (class)	30–45 min	Short command drill or flashcards; update next-step note
Wednesday	90–120 min	Main lab session; reproduce a concept and write findings
Thursday (class)	30–45 min	Light review or one small challenge
Friday	90–120 min	Independent challenge, debugging, or project work
Saturday	60–90 min	Weekly review, GitHub housekeeping, plan next week

Three levels for daily consistency

Level	Time	Good examples	Definition of success
Minimum day	10–15 min	Recall 5 terms; re-read yesterday's note; solve one small Bandit step; write a question to investigate	You stayed connected to the material. No guilt if you stop.
Normal day	45–75 min	20 min theory + 25–40 min lab + 5–10 min notes	One concept understood and evidenced.
Deep-study day	90–120 min	Theory, lab, independent repeat, debugging, short report	A result you can explain and reproduce.

Rules that protect the learning goal

- A streak is a reminder, not the objective. Never publish filler commits to keep a graph green.
- Do not require progress on THM, HTB, and GitHub every single day. One main learning platform per session is enough.
- Keep one active primary module and one small secondary practice stream. Avoid starting five paths at once.
- After a missed day, resume with the next planned step. Do not double the workload or restart the roadmap.
- During exams, switch to minimum days or pause deliberately; protect sleep and coursework.

4. Repeatable Study Method

The 7-step learning loop

Step	What to do
1. Orient	Write what the concept is, why it matters, and what you expect to observe.
2. Learn	Read one structured lesson; avoid opening many tabs. Record terms you cannot explain.
3. Inspect	Use official documentation to clarify how the protocol, command, or system behaves.
4. Practise guided	Complete a guided exercise and note what each action changes or reveals.
5. Reproduce	Close the walkthrough. Repeat from a blank state or solve a near-transfer task.

Step	What to do
6. Explain	Write a short explanation, include evidence/output, and state limitations.
7. Retrieve later	Review after about 1 day, 1 week, and 1 month; adjust intervals when recall is weak.

When to use hints, walkthroughs, and AI

- Try independently for 15–25 minutes after confirming the task and prerequisites. Write down what you tried.
- Use a hint to narrow the search space; return to the task and perform the reasoning yourself.
- Use a walkthrough after a serious attempt or when the lab teaches a prerequisite you lack. Read it in small chunks, close it, then reproduce the reasoning.
- Use AI to ask for a concept explanation, compare two hypotheses, or interpret a sanitized error. Ask for hints before full solutions.
- Never paste secrets, tokens, private target data, or prohibited lab content into public tools. Follow each platform's disclosure and write-up rules.
- If stuck for 30–45 minutes, step back: define the symptom, gather evidence, test one hypothesis, consult docs, then ask a narrower question.

Active recall prompts

- Explain the concept without notes in 60 seconds.
- Predict the output before running the command; then compare the actual output.
- What evidence supports your conclusion? What alternative explanation could fit?
- What would change if the service, OS, permissions, or network path were different?
- How would you fix or prevent the weakness, not just demonstrate it?

5. Phase 0 — Setup and Baseline (Week 1)

Goals

- Confirm the Mac's exact chip model and macOS version; check UTM VM architecture and current disk allocation.
- Create a clean notes repository and learning log.
- Capture current ability without judging it: Linux navigation, basic networking terms, one HTTP request, basic Python, Git status/commit.

UTM and storage plan

Keep your current Kali CLI VM if it works. Kali's official UTM instructions specifically document a console-only installation path for Apple Silicon, so a CLI VM is a reasonable choice. Do not reinstall simply because it lacks a desktop.

- Take a clean VM snapshot before major configuration changes, if your UTM version and VM setup support snapshots.
- Use NAT/shared networking for ordinary internet access. Use host-only or isolated networking for intentionally vulnerable targets when the platform supports it.
- Do not bridge an intentionally vulnerable VM onto a public or shared network. Never scan networks you do not own or have explicit permission to test.
- Keep only the tools you need. Browser-based THM/PortSwigger labs reduce local disk usage.
- With 20–50 GB free, avoid several large target VMs at once. Check VM disk usage and leave macOS breathing room.

Baseline checklist

- I can use pwd, ls, cd, cat, less, find, grep, head, tail, chmod, ps, and man/--help with basic confidence.
- I can explain an IP address, port, DNS lookup, TCP connection, and HTTP request at a high level.
- I can write a short Python program with variables, conditions, loops, functions, and file input/output.
- I can use git status, add, commit, log, and push without copying commands blindly.

- I know which lab targets are explicitly authorized.

6. Phase 1 — Linux and Operating-System Fundamentals (Weeks 2–6)

Order and rationale

Learn the shell and file model first; permissions and processes make more sense once you can navigate and inspect the system. Services and logs then prepare you to understand what network enumeration is actually discovering.

Sequence	Study and practise	Deliverable
1. Filesystem & shell	Paths, hidden files, globbing, pipes, redirection, quoting, help pages	A command notebook with examples and expected outputs
2. Users & permissions	Users/groups, rwx, ownership, sudo, umask, special bits at a conceptual level	Explain why a command succeeds or fails under two users
3. Processes & services	ps, top, signals, systemd/service concepts, listening sockets	Map a process to a service and its logs
4. Text & search	grep, sort, uniq, cut, awk basics, find, xargs carefully	Parse a sample log and explain each pipeline stage
5. OS fundamentals	Processes vs threads, memory, files, environment variables, packages, boot basics	One-page diagram of how a command runs
6. SSH & file transfer	SSH keys conceptually, host keys, scp/sftp, safe remote access	Connect to a legal practice target and document the steps

Free resources and assigned scope

- **HTB Academy — Linux Fundamentals:** continue your current module and finish all sections and exercises. All Tier 0 modules are free to complete under the Academy's current rules; higher-tier content may require Cubes or payment. academy.hackthebox.com
- **OverTheWire Bandit:** start at Level 0 and work through the early levels; focus on levels that reinforce file search, permissions, text processing, and SSH. Free. Bandit
- **Linux Journey:** use Grasshopper sections on command line, text-fu, permissions, processes, and packages as a second explanation. Free and browser-based. Linux Journey
- **pwn.college:** revisit the Linux Luminarium and Computing 101 dojos only for gaps; the platform describes itself as free for learners. pwn.college

Completion gate

- Given a new directory tree, find a named file, filter matching lines, and explain your pipeline.
- Explain a permission-denied error and safely diagnose it without reflexively using sudo.
- Identify a process and explain the difference between a process, a service, and a listening port.
- Complete a selected set of Bandit levels without reading the full solution first.
- Write a one-page Linux field guide in your own words.

7. Phase 2 — Networking and Traffic Reasoning (Weeks 7–11)

Learning order

- Addressing: IPv4, subnet basics, private/public addresses, MAC addresses, default gateway.
- Layering: Ethernet, IP, TCP/UDP, ports, sockets; what routing does and does not do.
- Name resolution: DNS records, resolver flow, caching, and common failure modes.
- Web transport: HTTP methods/status codes/headers/cookies; TLS purpose and certificate validation.

- Network tools: ping, ip/ifconfig, ss/netstat, dig/nslookup, curl, traceroute; understand limits and expected output.
- Packet inspection: capture your own traffic or use provided sample captures; identify DNS, TCP handshake, HTTP metadata, retransmissions at a beginner level.

Free resources

- **MDN — Overview of HTTP:** read the HTTP flow, request/response messages, headers, cookies, and statelessness sections. Free. MDN HTTP overview
- **Wireshark User's Guide:** begin with interface basics, opening captures, display filters, and following a TCP stream. Free documentation; Wireshark is free/open source. Wireshark guide
- **Nmap Network Scanning:** use the introduction and scan phases chapters as reference; the online guide is partly available for free. Practise only on your own VM, localhost, or authorized labs. Nmap guide
- **TryHackMe free rooms:** choose currently free rooms covering networking, DNS, HTTP, or Wireshark. The free-room catalogue changes; check the free label before starting. Free rooms catalogue

Practical exercise

From your own machine, look up a public domain's DNS records, request a public page with curl, and record the HTTP status and selected headers. Then inspect a provided or self-generated packet capture. Explain the difference between a DNS failure, TCP connection failure, TLS validation error, and HTTP 404. Do not capture other people's traffic.

Completion gate

- Draw the path from typing a domain name into a browser to receiving an HTTP response.
- Explain the difference between IP address, port, protocol, DNS name, and URL.
- Interpret a simple Nmap result without assuming that a port number proves which software is running.
- Use a packet capture to identify a DNS query and a TCP handshake, and explain what the evidence does and does not show.

8. Phase 3 — Bash, Python, Git, and Documentation (Weeks 12–15)

Keep scripting small and purposeful

Learn enough scripting to remove repetitive work and understand tool output. Do not write an exploit framework or large scanner yet. Every script should have a narrow purpose, safe defaults, readable errors, and a README.

Bash essentials

- Variables, quoting, exit status, stdin/stdout/stderr, pipes and redirection.
- if/for/while, functions, positional arguments, input validation, and safe handling of filenames.
- Use shellcheck or manual review where available; avoid destructive commands until you understand them.

Python essentials

- Refresh strings, lists/dicts, loops, functions, modules, exceptions, and reading/writing files.
- Learn requests/HTTP concepts only after networking and HTTP basics; parse data before automating network activity.
- Use argparse for command-line arguments and logging or clear print statements for understandable output.
- Keep dependencies minimal; document the Python version and setup.

Free resources

- **Official Python Tutorial:** use it as a reference for control flow, data structures, functions, files, errors, and modules; it assumes some programming familiarity, which suits your basic Python background. Free. Python tutorial
- **GitHub Skills — Introduction to GitHub:** complete the guided repository, branch, commit, and pull-request exercise. Free, about an hour. Introduction to GitHub

- **Pro Git:** use the free online book sections on getting started, recording changes, branches, and remotes. Pro Git
- **ShellCheck:** use it to catch common shell-script errors once you begin writing scripts. Free/open source. ShellCheck

Portfolio exercises

- Script 1: parse a supplied text log and summarize counts by status or keyword; no network scanning.
- Script 2: a small local environment checker that reports OS/Python version and checks whether required local files exist.
- Repository: README with purpose, prerequisites, safe usage, example input/output, limitations, and tests.
- Use branches for meaningful changes; commit related work with descriptive messages.

Completion gate

- Write and explain a 30–60 line script without copying a complete solution.
- Handle one invalid input and one expected error gracefully.
- Use Git to make a branch, commit a change, merge it, and explain what each command did.
- A classmate can follow the README and reproduce the project.

9. Phase 4 — Security Principles, Reconnaissance, and Enumeration (Weeks 16–20)

Method before tools

The purpose of reconnaissance is to reduce uncertainty. Enumeration gathers detail about reachable services and their behavior. A finding is a hypothesis supported by evidence—not merely a banner, a scanner label, or an open port.

Stage	Question	Evidence to record
Scope	What am I allowed to test?	Target, allowed actions, time limits, rules
Discover	What is reachable?	Addresses/hosts within scope; discovery method
Enumerate	What services and behaviors are exposed?	Ports, protocol responses, versions only when supported
Hypothesize	What might this imply?	Hypothesis plus alternative explanations
Validate	Can I test the hypothesis safely?	Minimal proof in the authorized lab
Report	What is the risk and fix?	Impact, evidence, reproducibility, remediation

Study topics

- Security principles: confidentiality, integrity, availability; authentication vs authorization; least privilege; attack surface; threat, vulnerability, exploit, risk, and impact.
- Common services: SSH, HTTP(S), DNS, SMB, FTP and database services at a conceptual level. Learn purpose, expected interactions, common misconfigurations, and safe verification.
- Enumeration: correlate Nmap output with protocol-specific evidence; research service documentation; distinguish facts from assumptions.
- Basic reporting: scope, summary, methodology, evidence, impact, remediation, and limitations.

Free practice

- **HTB Academy Tier 0:** prioritize any free modules available to you that directly reinforce networking, web requests, or Linux. Do not assume all Academy content is free; higher tiers may cost Cubes.
- **TryHackMe:** choose free beginner rooms in enumeration and basic pentesting; use the free-room catalogue to confirm access before committing.
- **Nmap official guide:** revisit scan types and service/version detection. Run scans only against your own lab or explicitly authorized targets.

Completion gate

- Produce a one-page assessment report for a beginner lab, with explicit scope and no unsupported claims.
- Explain why an open port is a starting point, not proof of a vulnerability.
- Given a service result, identify the next safe question and the documentation you would consult.
- Complete a simple enumeration lab without a walkthrough, then compare your notes with a solution only after the attempt.

10. Phase 5 — Web Technologies and Application Security (Weeks 21–27)

Your SQL injection re-learning plan

Because you have already completed SQL injection labs and watched Rana Khalil's explanations, do not simply replay the videos while copying payloads. Rebuild the mental model: how the application forms a query, how input changes query structure, what the response reveals, and how parameterized queries prevent the flaw.

- First: review HTTP requests, parameters, cookies, sessions, status codes, and Burp Suite Proxy/Repeater basics.
- Next: revisit PortSwigger SQL injection learning-path theory and Apprentice labs, starting with detection and retrieving hidden data.
- For each lab, write the observed behavior, the hypothesis, the minimal test, the evidence, and the secure fix.
- Then revisit boolean-based and error-based blind SQLi concepts; only move to time-based/out-of-band techniques after the basics make sense.
- Finally: write a small local-only Python helper for a toy application or supplied data that demonstrates HTTP request construction and response parsing. Do not build a tool for attacking arbitrary websites.

Broader web curriculum

Order	Topic	What to understand
1	Web foundations	HTTP request/response, methods, headers, cookies, sessions, TLS, same-origin basics
2	Authentication & access control	Login flows, session management, authorization checks, IDOR/access-control failures
3	SQL injection	Query context, UNION/error/blind concepts, parameterized queries, least-privilege DB accounts
4	XSS	Reflected/stored/DOM contexts, output encoding, sanitization, CSP as defense-in-depth
5	CSRF & file handling	Browser credentials, anti-CSRF defenses, upload validation, path traversal concepts
6	APIs & logic	JSON, API authorization, input validation, business logic, rate limits
7	Testing methodology	Map the app, form hypotheses, test one issue at a time, record evidence and remediation

Free resources and exact scope

- **PortSwigger Web Security Academy:** free online theory and interactive labs. Follow the SQL injection learning path: introductory material, detection, retrieving hidden data, subverting application logic, error-based and blind SQLi, then prevention. Start with Apprentice labs and mark Practitioner as stretch. SQL injection learning path
- **PortSwigger topic index:** after SQLi, do selected Apprentice labs for XSS, access control, authentication, and CSRF. Free. Web Security Academy
- **OWASP Web Security Testing Guide:** use its information-gathering, configuration, identity, authentication, authorization, session, input-validation, and business-logic sections as a checklist—not as a requirement to master the entire guide now. Free. OWASP WSTG
- **MDN HTTP overview:** revisit whenever request/response behavior is unclear.

Completion gate

- Re-solve at least 8–12 selected SQLi labs across basic categories without copying payloads; explain why each test works.
- For each lab, identify a defensible remediation, especially parameterized queries for SQLi.
- Complete at least 2 Apprentice labs in one additional web category such as XSS or access control.
- Write a concise, policy-compliant lab report that does not disclose prohibited platform solutions.

11. Phase 6 — Windows, Privilege Concepts, and Defensive Fundamentals (Weeks 28–34)

Windows before Windows exploitation

Learn how Windows represents users, groups, permissions, processes, services, scheduled tasks, PowerShell, and event logs. Only then study why excessive privileges or insecure service configurations matter. Use a free browser lab or an appropriately licensed local image; do not download random Windows images from unofficial sources.

Linux and Windows privilege escalation concepts

- Understand the difference between a normal user and an elevated account; learn why privilege boundaries exist.
- Learn common misconfiguration classes at a conceptual level: writable scripts/configurations, excessive permissions, exposed credentials, unsafe service/task settings, and outdated software.
- Practise enumeration before exploitation: current identity, groups, privileges, running processes/services, file permissions, and relevant logs.
- Record a finding only when the lab evidence supports it; distinguish a suspicious configuration from a proven path.

Defensive basics

- Read authentication and service logs; build a timeline from timestamps and event descriptions.
- Use Wireshark to inspect a provided capture and explain what is normal, unusual, and uncertain.
- Practise least privilege, updates, secure configuration, backups, logging, and credential hygiene.
- Introduce incident-response thinking: identify, preserve evidence, scope, contain, recover, and document—at an introductory level.

Free resources

- **Microsoft Learn:** search its free Windows client, PowerShell, users/permissions, and event-log learning material; use the specific module relevant to the concept you're practising. Microsoft Learn
- **Wireshark User's Guide:** continue with display filters and following streams.
- **TryHackMe free rooms:** choose currently free Windows fundamentals or defensive/log-analysis rooms from the free-room catalogue; availability changes.
- **pwn.college:** revisit relevant systems dojos if process execution or memory concepts are a prerequisite for your next topic.

Completion gate

- Explain user, group, permission, process, service, and scheduled-task concepts in Windows.
- Inspect a sample log and produce a short evidence-based timeline.
- Explain one Linux and one Windows privilege misconfiguration and how to prevent it.
- Complete one beginner host-focused lab with minimal hints and document the methodology.

12. Phase 7 — Integration, CTF Independence, and Capstone (Weeks 35–42)

CTF practice method

- Start with Bandit and the introductory pwn.college dojos you have not mastered; use beginner THM rooms only when free.

- Choose a challenge that matches current prerequisites. Do not use a complex machine as a substitute for learning networking or Linux.
- For each attempt, maintain a time-boxed log: objective, known facts, hypotheses, commands, observations, dead ends, and next step.
- After solving, wait a day and reproduce the key steps without the walkthrough. If you cannot explain the path, count it as exposure—not mastery.

Suggested capstone: Local Lab Assessment Notebook

Create a portfolio project that demonstrates your process rather than a flashy exploit. Use only a deliberately vulnerable local application or an explicitly authorized training lab.

- Scope and lab diagram; note VM architecture and network mode.
- Discovery and enumeration notes with commands, selected outputs, and interpretations.
- Two or three verified findings, each with impact and remediation.
- A defensive section: what logs or traffic might show, and what additional evidence would be needed.
- One small helper script that processes local text/JSON output; include tests and limitations.
- A clean README, screenshots only where allowed, references, and a statement that the work was performed in an authorized lab.

Final readiness gate

- I can explain a browser-to-server connection and basic DNS/TCP/HTTP behavior.
- I can navigate Linux, search/filter text, inspect processes, and reason about permissions.
- I can write and debug small Bash/Python scripts and use Git safely.
- I can follow a scope-first enumeration process and distinguish evidence from assumptions.
- I can explain SQLi, XSS, and access-control weaknesses plus practical defenses.
- I can read basic host logs or a packet capture and describe uncertainty honestly.
- I can solve a beginner challenge with limited hints, then reproduce the reasoning later.
- My public portfolio contains no secrets, private data, prohibited write-ups, or platform flags where publication is disallowed.

13. Resource Directory and Free-Access Notes

The links below are official or project-maintained starting points. 'Free' means the linked learning content is currently available at no charge; it does not mean every feature, lab, certification, hosted environment, or related course on that platform is free.

Resource	Best use in this plan	Access note
HTB Academy	Continue Linux Fundamentals; select Tier 0 modules relevant to current phase	Tier 0 modules are free to complete under current Academy rules. Higher tiers may require Cubes/payment.
TryHackMe free rooms	Networking, enumeration, beginner pentesting, Windows/defense when marked free	Free account and free rooms; some paths, rooms, and unlimited AttackBox access require paid tier.
PortSwigger Web Security Academy	SQLi, XSS, access control, authentication, CSRF	Free interactive labs and learning material.
OverTheWire Bandit	Linux shell, files, search, SSH, problem-solving	Free.
pwn.college	Linux Luminarium, Computing 101, systems/security dojos	Platform describes itself as free for learners; some course modules can be time/cohort structured.
Linux Journey / LabEx	Second explanation for Linux basics	Current Linux Journey site says free and no sign-up required; site is now hosted by LabEx.
MDN HTTP	Web requests/responses, headers, cookies, HTTP behavior	Free documentation.

Resource	Best use in this plan	Access note
Wireshark User's Guide	Traffic inspection and filters	Free documentation; Wireshark is free/open source.
Nmap official guide	Scan concepts, service detection, interpretation	Free tool; online guide is partly available for free.
Python Tutorial	Refresh Python language features	Free official documentation.
GitHub Skills	Repository, branch, commit, pull request	Free guided exercise; GitHub account needed.
Pro Git	Git fundamentals and branching	Free online book.
OWASP WSTG	Web testing methodology and defensive thinking	Free open project and documentation.
Microsoft Learn	Windows and PowerShell fundamentals	Free learning modules; some sandbox/product services may have limits.
OWASP Juice Shop	Optional local web-security practice after setup confidence	Free/open source, but local installation uses storage and may require Node.js or Docker.

Direct links

• **HTB Academy** — https://academy.hackthebox.com/

• **HTB Academy access/pricing explanation** — https://help.hackthebox.com/en/articles/12256202-introduction-to-htb-academy

• **TryHackMe free rooms** — https://tryhackme.com/free-rooms

• **TryHackMe pricing/access** — https://tryhackme.com/pricing

• **PortSwigger Web Security Academy** — https://portswigger.net/web-security

• **PortSwigger SQL injection learning path** — https://portswigger.net/web-security/learning-paths/sql-injection

• **OverTheWire Bandit** — https://overthewire.org/wargames/bandit/

• **pwn.college** — https://pwn.college/

• **pwn.college future dojo catalogue** — https://future.pwn.college/

• **Linux Journey** — https://labex.io/linuxjourney

• **Kali Linux in UTM official guide** — https://www.kali.org/docs/virtualization/install-utm-guest-vm/

• **MDN HTTP overview** — https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview

• **Wireshark User's Guide** — https://www.wireshark.org/docs/wsug_html/

• **Nmap Network Scanning** — https://nmap.org/book/

• **Python Tutorial** — https://docs.python.org/3/tutorial/

• **GitHub Skills: Introduction to GitHub** — https://github.com/skills/introduction-to-github

• **Pro Git** — https://git-scm.com/book/en/v2

• **ShellCheck** — https://www.shellcheck.net/

• **OWASP Web Security Testing Guide** — https://owasp.org/www-project-web-security-testing-guide/

• **Microsoft Learn** — https://learn.microsoft.com/training/

• **OWASP Juice Shop** — https://owasp.org/www-project-juice-shop/

14. GitHub Portfolio Structure

Start with one repository. Split into multiple repositories only when a project has a distinct purpose or would be useful to present independently.

```
cybersecurity-learning/
■■■ README.md
■■■ learning-log/weekly-review.md
■■■ notes/linux/
■■■ notes/networking/
■■■ notes/web-security/
■■■ labs/authorized-lab-template.md
```

```
■■■ scripts/log_parser/
■■■ projects/local-lab-assessment/
■■■ references/commands.md
```

Lab note template

Field	What to record
Title/date	Topic, platform, date, time spent
Scope	Authorized target and applicable platform rules; avoid sensitive target details
Goal	Question you are trying to answer
Prerequisites	Concepts/tools required
Method	Steps in your own words; commands with flags explained
Evidence	Small sanitized output excerpts and what each one means
Reasoning	Hypothesis, alternatives, conclusion, uncertainty
Lessons	Mistakes, root cause, what to review
Next review	One-day, one-week, and one-month recall dates

Public portfolio hygiene

- Never commit API keys, passwords, tokens, private keys, cookies, personal data, or real target details.
- Respect platform rules on flags, screenshots, and write-ups; when unsure, publish concepts and defensive lessons rather than a solution.
- Use .gitignore for local captures, VM images, environment files, large dumps, and generated artifacts.
- Use a clear README: purpose, prerequisites, setup, safe use, examples, limitations, references, and license where appropriate.
- Prefer meaningful commits tied to a real change. A weekly summary is useful; daily commits are not mandatory.

15. Troubleshooting and Getting Unstuck

Symptom	Process
Command output differs from a guide	Check OS/shell/version, current directory, permissions, quoting, and tool version. Compare documentation before changing system settings.
Network request fails	Separate DNS resolution, routing, TCP connection, TLS validation, HTTP status, and application behavior. Test one layer at a time.
A lab feels impossible	Re-state the objective; list known facts; identify missing prerequisite; choose one test that distinguishes two hypotheses.
VM is slow or storage is low	Stop unused VMs, check allocated vs actual disk use, remove unneeded downloads, use browser labs, and avoid extra desktop environments unless needed.
A walkthrough solves it instantly	Close it and recreate the reasoning from scratch. Write down the clue you missed and make a similar practice question.
Repeatedly forgetting commands	Use a command reference with purpose, example, expected output, and common failure—not a raw list of flags.
You have missed several days	Do a 10-minute review, choose the next smallest task, and resume. Do not restart the whole roadmap.

Command reference template

For each command, record: purpose; a safe example; important options; sample output; interpretation; common errors; whether it changes the system; and a link to official documentation. For security tools, add a scope/safety note.

Debugging questions

- What exactly did I expect, and what actually happened?
- What changed since the last successful attempt?
- Can I reproduce it with the smallest possible example?
- What evidence would distinguish my top two explanations?
- Have I checked the manual, version, permissions, environment, and platform rules?

16. Weekly Review, Tracker, and Maintenance

Weekly review — 15 to 25 minutes

- What concept can I explain without notes now?
- What did I practise independently rather than follow?
- Which output did I interpret incorrectly, and why?
- What is the one unresolved gap that blocks the next phase?
- What is next week's single primary goal?
- What should be spaced-repeated tomorrow, next week, and next month?

Simple tracker

Week	Primary topic	Theory	Lab repeated independently	Notes/project	Recall date	Confidence (1–5)
1	Setup + baseline	■	■	■	_____	___
2	Linux shell	■	■	■	_____	___
3	Permissions/processes	■	■	■	_____	___
4	Linux troubleshooting	■	■	■	_____	___
5	Networking	■	■	■	_____	___
6	HTTP/DNS	■	■	■	_____	___
7	Scripting/Git	■	■	■	_____	___
…	Continue with one row per week	■	■	■	_____	___

When to revise the roadmap

Review this plan every four weeks. If you pass the completion gate easily twice, move forward. If you fail, identify the missing sub-skill and add one or two focused sessions. Do not restart the entire phase. If your interest changes, complete the core foundation first, then replace some integration weeks with a specialization sampler.

After the core: choose a specialization

Path	First sampler after the foundation	Build toward
Penetration testing	More service enumeration, methodology, reporting, Linux/Windows host labs	Scoped assessments, repeatable notes, clear remediation
Web application security	More PortSwigger labs, access control, auth, APIs, secure coding	Web testing methodology and responsible disclosure
SOC / blue team	Log analysis, Windows events, Wireshark, detection logic, incident timelines	Alert triage, investigation notes, basic detection engineering
Systems / low-level security	pwn.college systems dojos, C basics, memory/process fundamentals	Debugging and exploit-development prerequisites
Cloud security	Networking, identity, Linux, web/API basics first; then free official cloud training	Cloud IAM, logging, network boundaries, secure configuration

17. Final Checklist and First Seven Days

Your first week

Day	Task	Output
1	Confirm Mac model/macOS, UTM architecture and disk usage; create notes repo	Environment note + repo README
2	Continue HTB Linux Fundamentals; write 5 active-recall questions	Module notes, no copied paragraphs
3	Practise file navigation, search, permissions, and pipelines	10-command reference with interpretations
4	Review a Bandit level or repeat a previous THM enumeration exercise	Lab note with hypothesis and evidence
5	Read MDN HTTP overview; explain request/response in your own words	One-page HTTP summary
6	Repeat one earlier task without the walkthrough	Record where you got stuck and why
7	Weekly review and choose next week's target	A realistic next-step plan

Ready to progress when…

- I can explain the main ideas without simply reciting a tool's output.
- I can predict what a command or request is meant to do before running it.
- I can use documentation and troubleshoot basic failures.
- I can repeat a lab independently or clearly identify the specific prerequisite I lack.
- I can record evidence, explain uncertainty, and describe a safe remediation.
- My learning pace is sustainable alongside university and rest.

Remember: the goal is durable understanding. A slower lab that you can explain next week is more valuable than five badges you cannot reproduce.

18. Sources, Verification, and Important Caveats

This plan uses official platform pages and project documentation for access claims and learning scope. Access labels were checked on 14 October 2026. Platforms can change pricing, room availability, account requirements, and publication rules; check the linked official page before each new module.

- HTB Academy help pages state that Tier 0 modules are free to complete, while other tiers use Cubes and may require payment.
- TryHackMe distinguishes free rooms from paid learning paths, premium access, and AttackBox limits. Select only rooms explicitly marked free.
- PortSwigger Web Security Academy describes its training and interactive labs as free.
- OverTheWire Bandit is designed for beginners and practises Linux command-line problem-solving.
- pwn.college describes its educational platform as free to participate in; individual dojos may be organized around course iterations.
- Kali's official UTM guide includes an Apple Silicon-oriented console-only installation path; compatibility still depends on the exact Mac/UTM/Kali versions.
- OWASP Juice Shop is an optional intentionally vulnerable application. Because local setup consumes storage and may involve Docker/Node.js, it is not required in the early phases.

Safety boundary: only test systems you own or have explicit permission to assess. Keep vulnerable targets isolated, obey lab rules, and do not publish credentials, flags, sensitive data, or write-ups prohibited by the platform.

This document is a learning plan, not a certification syllabus or guarantee of professional competence. Use it as a living guide: review, measure, and revise it as your skills and university workload change.

End of roadmap • Start with Phase 0, then continue the Linux Fundamentals module you are already working on.